

INSTITUTO DE INTELIGÊNCIA ARTIFICIAL APLICADA — I2A2

Protege Antes Ferramenta Inteligente para Comunicação Proativa com o Segurado

Relatório técnico · Desafio 5

Grupo: Cortex

Representante: Marcelo Barros de Azevedo Vieira — marcelobazevedo@gmail.com

Integrantes:

Fabício Oliveira Magalhães de Barros — fabriciotrabalho01@gmail.com

Luiz Eduardo Santos Lopes — dgjheuh@gmail.com

Alan Heverton Lima Martins — alanheverton7@gmail.com

Versão: 1.0

Data: agosto de 2026

Licença: MIT

Natureza: protótipo educacional com segurados fictícios e previsão real

1. Resumo executivo

O Protege Antes é um MVP que transforma o relacionamento entre seguradora e segurado de reativo em preventivo. O sistema consulta previsões meteorológicas externas, identifica eventos potencialmente danosos, cruza o risco com a localização e as apólices dos segurados, gera orientações personalizadas com um modelo de linguagem e entrega alertas pelo canal configurado.

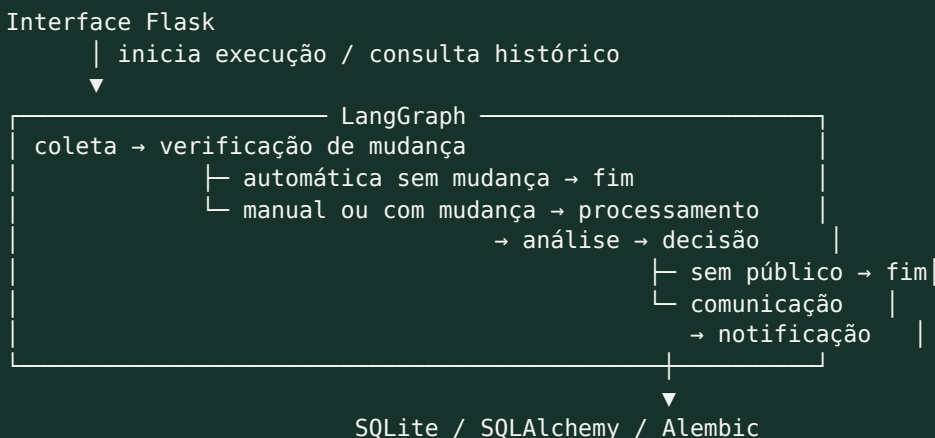
A solução demonstra um fluxo multiagente com LangGraph, modelo local Ministral 3 por Ollama, integração configurável com a Groq, persistência SQLite com SQLAlchemy e Alembic, interface Flask e ambiente Docker. A integração com a Evolution API realiza envio real por WhatsApp quando ativa; SMS e e-mail permanecem simulados. A previsão é obtida da API pública Open-Meteo; segurados e exemplos de mensagens são fictícios.

2. Objetivos e escopo

O objetivo principal é antecipar riscos meteorológicos e comunicar ações preventivas antes de um sinistro. O MVP cobre chuva intensa, vento forte e granizo, seguros residencial e automóvel, previsão real pelo Open-Meteo e execução automática orientada a mudanças.

Fora do escopo: integração com sistemas reais de seguradoras, canais reais de SMS/e-mail, interpretação contratual de cobertura, operação comercial e cobertura de todos os eventos climáticos.

3. Arquitetura da solução



Fontes: Open-Meteo · Ollama local · Groq remoto opcional

A separação por camadas impede que interface, regras, integrações e persistência sejam misturadas. O Flask apresenta e inicia o caso de uso; o LangGraph coordena o estado; regras de domínio determinam o risco; adaptadores tratam APIs e banco.

3.1 Estado do grafo

O estado tipado contém identificador da execução, segurados, previsões, indicador de mudança, eventos, destinatários, mensagens e erros. Em execuções automáticas, uma aresta encerra o fluxo quando a previsão não mudou; execuções manuais sempre continuam. Outro ramo encerra quando não há público elegível, evitando chamadas desnecessárias à LLM.

3.2 Fluxo detalhado do grafo LangGraph

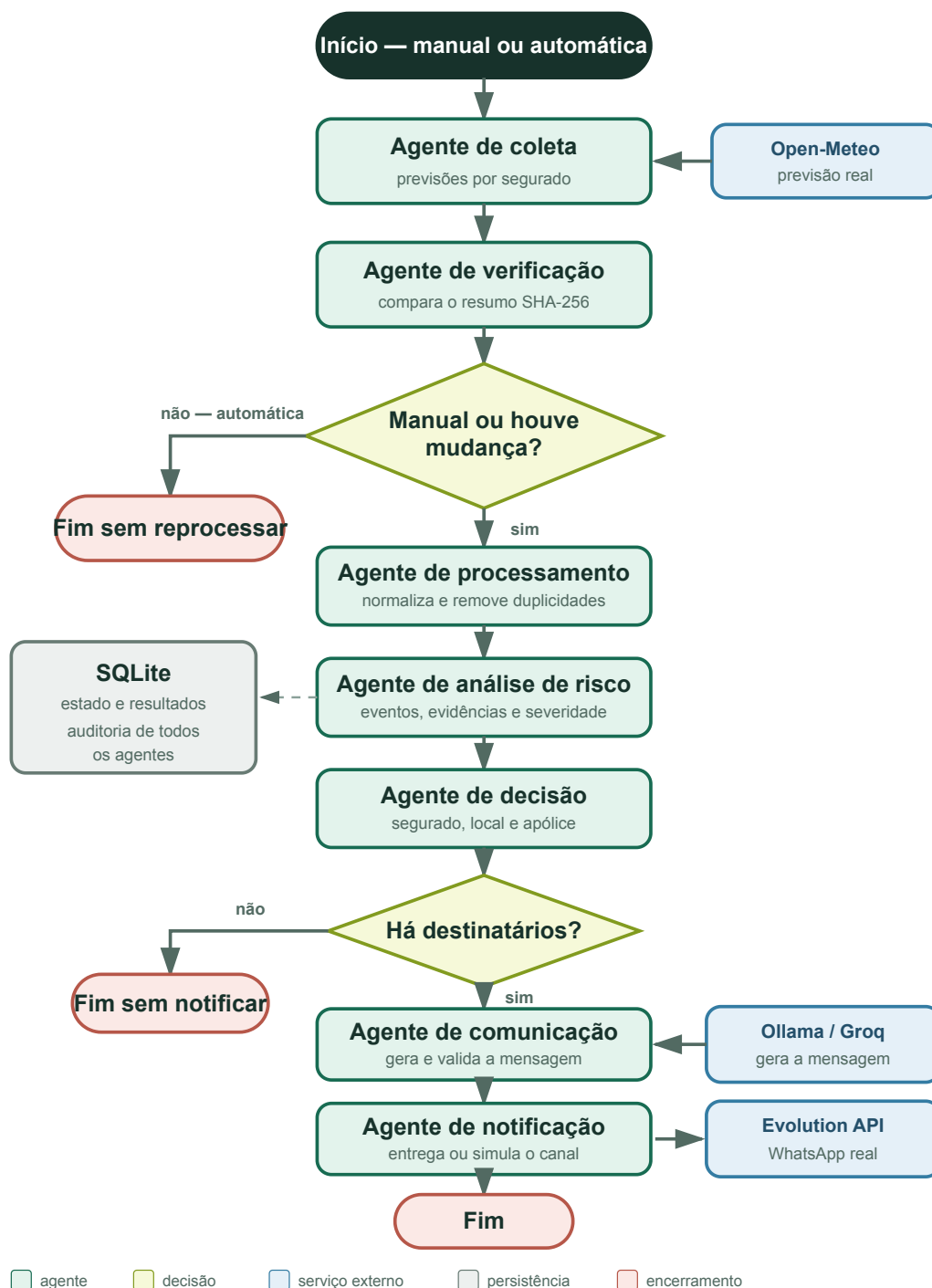


Figura 1 — Nós, decisões condicionais, integrações e persistência do grafo.

O LangGraph inicia na coleta e transporta um estado tipado entre os agentes. O primeiro desvio condicional evita trabalho repetido apenas no monitor automático: se a previsão não mudou, a execução termina; o botão manual sempre segue para processamento. Depois das regras de risco, o segundo desvio verifica se há segurados elegíveis. Sem destinatários, a execução termina sem chamar a LLM. Com destinatários, Ollama ou Groq gera o conteúdo, e o agente de notificação registra a simulação de SMS/e-mail ou entrega o WhatsApp pela Evolution API.

A LLM não determina riscos nem escolhe segurados. Essas decisões são determinísticas e acontecem antes da comunicação. Todos os nós registram entrada, saída, duração, status e eventuais erros no SQLite, formando a trilha de auditoria apresentada pela interface.

4. Agentes especializados

Agente	Responsabilidade	Saída
Coleta	Consulta a previsão real no Open-Meteo.	Previsões associadas aos segurados.
Verificação	Compara o resumo SHA-256 normalizado com a última coleta.	Indicador auditável de mudança.
Processamento	Normaliza e elimina registros duplicados.	Previsões internas consistentes.
Análise de risco	Aplica limiares meteorológicos determinísticos.	Eventos, severidade e evidência.
Decisão	Cruza evento, localidade e tipo de apólice.	Decisões positivas e negativas justificadas.
Comunicação	Gera e valida JSON estruturado com Ollama/Groq.	Mensagem personalizada.
Notificação	Entrega WhatsApp pela Evolution API ou simula SMS/e-mail.	Resultado auditável no SQLite.

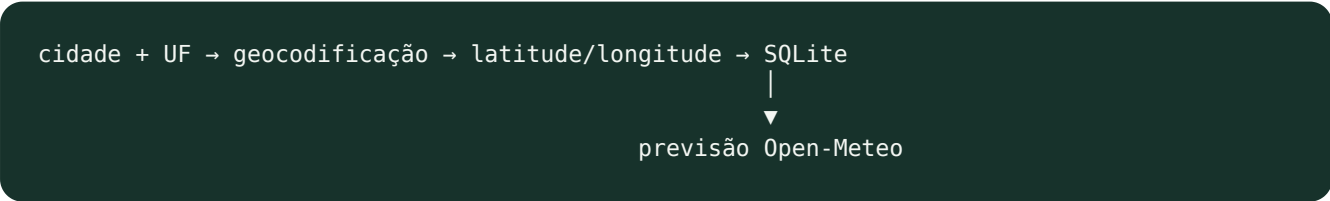
A LLM não decide se existe risco nem quem será notificado. Essas decisões permanecem explicáveis e testáveis nas regras de domínio.

5. Tecnologias utilizadas

Tecnologia	Uso
Python 3.13 e uv	Aplicação, dependências reproduzíveis e comandos.
LangGraph	Estado, nós, arestas e ramificação do fluxo.
Ollama + ministral-3:14b	Geração local de mensagens quando LOCAL=True.
Groq + openai/gpt-oss-20b	Alternativa remota configurável quando LOCAL=False; o uso real requer uma chave da Groq.
Open-Meteo	Previsão diária e geocodificação de cidade/UF.
Flask e Jinja	Painel, cadastro e auditoria visual.
SQLite, SQLAlchemy e Alembic	Persistência, ORM e evolução do esquema.
Docker Compose	Ambiente de desenvolvimento com volumes e código montado.
Evolution API 2.3.7, PostgreSQL 16 e Redis 7	Envio real por WhatsApp, sessão, persistência e cache.
Ruff e Pyright	Análise estática e verificação de tipos.

6. Fontes externas e geocodificação

O usuário cadastra nome, e-mail, telefone com DDD, cidade, UF, apólices e canal preferido. O telefone é normalizado com o código do Brasil para permitir a entrega pelo WhatsApp quando a Evolution API estiver habilitada. A API de geocodificação do Open-Meteo procura a cidade no Brasil, valida a unidade federativa e retorna nome canônico, latitude e longitude. As coordenadas são persistidas e usadas pela API de previsão, que exige localização geográfica. Cada card permite editar nome, cidade, UF e apólices. Uma mudança de cidade ou UF refaz a geocodificação e atualiza as coordenadas. A exclusão lógica exige confirmação: o registro fica inativo e não participa de novas análises, enquanto notificações e execuções antigas permanecem auditáveis.



A previsão contempla até três dias. Depois da coleta, registros ordenados e sem duplicidade formam um resumo SHA-256. Se ele não mudou, o fluxo termina antes das regras, da LLM e das notificações.

7. Regras de negócio

Evento	Regra do MVP	Severidade	Apólices
Chuva intensa	≥ 50 mm, ou ≥ 30 mm com probabilidade $\geq 80\%$	Alta a partir de 50 mm	Residencial e automóvel
Vento forte	Rajada ≥ 75 km/h	Alta a partir de 90 km/h	Residencial e automóvel
Granizo	Código meteorológico WMO 96 ou 99	Alta	Automóvel

Os limiares são premissas transparentes do protótipo e não recomendações operacionais. Uma implantação real exigiria homologação por meteorologia, atuária e área de seguros.

Todos esses valores são carregados do `.env`: acumulados e probabilidade de chuva, limiares de vento e códigos WMO de granizo. A configuração é validada e convertida em um objeto imutável, injetado no agente de análise. Portanto, os parâmetros podem ser calibrados sem editar a regra de domínio ou reconstruir a imagem Docker.

7.1 Parâmetros configuráveis no ambiente

Variável do <code>.env</code>	Padrão	Descrição
ALERTA_CHUVA_INTENSA_MM	50 mm	Acumulado diário que dispara diretamente chuva intensa e severidade alta.
ALERTA_CHUVA_COM_PROBABILIDADE_MM	30 mm	Acumulado mínimo para alerta moderado quando combinado à probabilidade mínima.
ALERTA_PROBABILIDADE_MINIMA_CHUVA	80%	Probabilidade exigida para usar o limiar condicionado de precipitação.
ALERTA_VENTO_FORTE_KMH	75 km/h	Rajada mínima que caracteriza evento de vento forte.
ALERTA_VENTO_SEVERIDADE_ALTA_KMH	90 km/h	Rajada a partir da qual o vento forte recebe severidade alta.
ALERTA_CODIGOS_WMO_GRANIZO	[96,99]	Lista JSON de códigos meteorológicos WMO interpretados como trovada com granizo.

Os acumulados e as velocidades devem ser positivos; a probabilidade deve permanecer entre 0 e 100. O limiar condicionado de chuva não pode superar o de chuva intensa, e o limiar de severidade alta do vento não pode ser inferior ao de vento forte. Uma combinação inválida interrompe a inicialização com mensagem explícita, evitando decisões silenciosamente incoerentes.

```
# Exemplo no .env
ALERTA_CHUVA_INTENSA_MM=50
ALERTA_CHUVA_COM_PROBABILIDADE_MM=30
ALERTA_PROBABILIDADE_MINIMA_CHUVA=80
ALERTA_VENTO_FORTE_KMH=75
ALERTA_VENTO_SEVERIDADE_ALTA_KMH=90
ALERTA_CODIGOS_WMO_GRANIZO=[96,99]

# Aplicar alterações sem rebuild
docker compose up -d --force-recreate aplicacao
```

8. Geração das mensagens

A configuração LOCAL seleciona o provedor. O prompt solicita português brasileiro, texto curto, tom empático, recomendação prática, ausência de alarmismo e nenhuma promessa de cobertura. Ollama e Groq recebem JSON Schema, e Pydantic valida a resposta. As tentativas são configuráveis; se o provedor falhar, uma mensagem de contingência é produzida e a ocorrência fica registrada. A integração com a Groq está implementada e possui validação estruturada da resposta, mas uma chamada externa real depende do fornecimento de uma chave válida.

As mensagens abaixo são exemplos ilustrativos produzidos para documentar o comportamento esperado. Os nomes são fictícios; em operação, as condições meteorológicas vêm do Open-Meteo.

Exemplo — granizo e seguro automóvel

Olá, Ana! Há previsão de granizo em São Paulo em 19/08/2026. Se possível, guarde o veículo em local coberto e evite permanecer próximo a árvores. Esta é uma orientação preventiva e não representa confirmação de cobertura. Em emergência, siga as autoridades locais.

Exemplo — chuva e seguro residencial

Olá, Ana! Há previsão de chuva intensa em São Paulo em 19/08/2026, com acumulado estimado de 62 mm. Proteja objetos em áreas baixas e não atravesse vias alagadas. Em emergência, acompanhe as orientações dos órgãos oficiais.

9. Persistência e auditoria

O SQLite contém as tabelas segurados, apolices, execucoes, eventos_climaticos, mensagens, notificacoes, verificacoes_meteorologicas, consultas_meteorologicas, decisoes e auditorias_agentes. As migrações 0001 a 0003 criam a base, a localização e a exclusão lógica; a 0004 registra as verificações; a 0005 acrescenta o canal da notificação; e a 0006 normaliza a origem; a 0007 acrescenta preferências e auditoria; a 0008 separa mensagens; e a 0009 adiciona os contatos do

segurado e o destino da notificação; e a 0010 acrescenta idempotência, tentativas, resposta, erro e identificador externo do WhatsApp. O painel apresenta totais, histórico, eventos, evidências, destinatário, mensagem, canal, horário, provedor e status de entrega. O botão **Excluir todas** remove em uma transação notificações, eventos, verificações e execuções, após confirmação, mas preserva segurados e apólices. A limpeza é bloqueada enquanto houver uma execução em andamento.

10. Interface e operação

1. Acessar o painel em `http://localhost:5001`.
2. Consultar, cadastrar ou editar um segurado por cidade e UF.
3. Usar **Verificar agora** para consultar a previsão real.
4. Inspecionar a mudança, os riscos e as mensagens na auditoria da execução.
5. Consultar as execuções automáticas; ausência de mudança ou alertas é um resultado válido.

A ausência de mudança encerra antecipadamente apenas uma execução automática. Na execução manual, a pipeline completa é percorrida. A ausência de eventos acima dos limiares é sempre um resultado válido e encerra o ramo sem criar notificações desnecessárias.

11. Instalação a partir do GitHub

As instruções abaixo partem de um computador novo. São necessários Git, Docker Engine com docker compose, Ollama, curl e openssl. Linux é o ambiente recomendado porque aplicação e monitor usam a rede do host para acessar o Ollama. O uv é instalado na imagem e só é obrigatório no host para a execução alternativa sem Docker.

11.1 Clone e configuração

```
git clone https://github.com/marcelobazevedo/comunicacao-proativa.git
cd comunicacao-proativa
cp .env.exemplo .env
openssl rand -hex 32
```

O resultado do último comando deve ser diferente para CHAVE_SECRETA, EVOLUTION_API_KEY, EVOLUTION_POSTGRES_SENHA e EVOLUTION_REDIS_SENHA. O arquivo .env é ignorado pelo Git. Um clone novo não contém banco SQLite, volumes, credenciais ou sessão pareada do WhatsApp.

```
# Configuração inicial essencial no .env
LOCAL=True
OLLAMA_URL=http://localhost:11434
OLLAMA_MODEL=minstral-3:14b
ENVIOWHATSAPP_ATIVO=False
EVOLUTION_API_URL=http://127.0.0.1:8080
EVOLUTION_API_INSTANCIA=protege-antes
```

11.2 Ollama e inicialização completa

```
ollama pull minstral-3:14b
ollama list
curl http://localhost:11434/api/tags

docker compose --profile whatsapp up --build -d
docker compose --profile whatsapp ps
docker compose logs -f aplicacao
```

O perfil whatsapp inicia Flask, monitor, Evolution API 2.3.7, PostgreSQL 16 e Redis 7. O contêiner executa uv sync, aplica as migrações Alembic e cria os dados iniciais. Quando aplicacao estiver saudável, o painel estará em http://localhost:5001. Ollama e o modelo permanecem no host; código, SQLite e demais dados são preservados em volumes e alterações de fonte aparecem sem rebuild.

11.3 Evolution Manager e pareamento

1. Acessar http://localhost:8080/manager/login.
2. Informar http://localhost:8080 como URL e usar o valor de EVOLUTION_API_KEY como chave global.

3. Criar ou abrir a instância definida por EVOLUTION_API_INSTANCIA, normalmente protege-antes.
4. No celular, abrir WhatsApp → Dispositivos conectados → Conectar dispositivo e ler o QR Code.

```
# Diagnóstico sem envio
docker compose exec aplicacao uv run comunicacao-proativa validar-evolution

# Teste explícito para número autorizado
docker compose exec aplicacao uv run comunicacao-proativa testar-whatsapp \
  --numero "11999999999" \
  --mensagem "Teste de integração do Protege Antes."
```

O diagnóstico deve retornar estado: open. Após o teste, definir ENVIO_WHATSAPP_ATIVO=True no .env e recarregar os processos:

```
docker compose up -d --force-recreate aplicacao monitor
```

11.4 Validação, operação e recuperação

```
docker compose exec aplicacao uv run alembic current
docker compose exec aplicacao uv run ruff check .
docker compose exec aplicacao uv run pyright
docker compose logs -f monitor
docker compose --profile whatsapp down # preserva volumes
```

O resultado de referência é Alembic 0010 (head), Ruff e Pyright sem erros. Se o painel não abrir, conferir `docker compose --profile whatsapp ps` e os logs de aplicacao. Se o Ollama falhar, confirmar o serviço e `curl http://localhost:11434/api/tags`. Se o WhatsApp não enviar, executar `validar-evolution`, conferir o estado open e parear novamente no Manager. `docker compose --profile whatsapp down --volumes` apaga permanentemente SQLite, PostgreSQL, Redis, dependências e sessão do WhatsApp e só deve ser usado para reinicialização total.

11.5 Monitoramento e entrega de notificações

O monitor consulta imediatamente ao iniciar e repete a verificação a cada 30 minutos por padrão. MONITORAMENTO_ATIVO liga ou desliga o serviço lógico, e INTERVALO_MONITORAMENTO_MINUTOS ajusta a frequência. CANAL_NOTIFICACAO aceita whatsapp, sms ou email; a auditoria exibe canal, destino, tentativas, status e horário. WhatsApp usa a Evolution API quando ENVIO_WHATSAPP_ATIVO=True; SMS e e-mail permanecem simulados. O histórico marca como **Manual** as execuções iniciadas por botão ou CLI e como **Automática** as iniciadas pelo monitor. Execuções manuais percorrem a pipeline completa; somente as automáticas terminam antecipadamente quando não há mudança. A migração 0006 normaliza os registros anteriores para essa classificação. O painel aberto consulta novamente seu estado a cada 15 segundos; uma execução manual exibe um indicador de atividade e bloqueia cliques duplicados.

Falhas temporárias no Open-Meteo recebem até três tentativas. Uma falha parcial produz o status **concluída com ressalvas** e preserva o processamento das demais cidades; ausência total de previsões produz **erro**. O prompt pede texto simples, e a saída da LLM é normalizada para remover marcadores Markdown antes de persistir e exibir.

Todos os instantes são persistidos em UTC. Na apresentação, um filtro central interpreta como UTC os valores sem fuso devolvidos pelo SQLite e converte execuções e notificações para America/Sao_Paulo. A correção também alcança o histórico sem reescrever o banco.

Cada formulário POST possui token CSRF. Cada segurado escolhe WhatsApp, SMS ou e-mail. A auditoria exibe a linha do tempo dos agentes, as decisões negativas e positivas e cada tentativa de consulta meteorológica, incluindo fonte, parâmetros, duração, status e erro.

12. Verificações e resultados

A qualidade é verificada por Ruff, Pyright e validações funcionais. O funcionamento pode ser acompanhado pela interface, pelos logs e pela trilha persistida no SQLite.

Ruff	sem erros
Pyright	sem erros
Alembic	0010 (head)
Flask	HTTP 200
Ollama no host	ministral-3:14b acessível pelos contêineres
Evolution API	2.3.7 saudável; instância open; envio HTTP 201 confirmado
Groq remoto	integração implementada; chave externa não incluída

13. Segurança, ética e limitações

- Os dados iniciais e exemplos de mensagens são fictícios; a validação controlada do WhatsApp usou um destino real sem expô-lo no relatório. O `.env` não é versionado e formulários usam CSRF.`
- Mensagens por WhatsApp podem ser reais quando habilitadas e não substituem Defesa Civil ou órgãos meteorológicos.
- O sistema não promete cobertura ou indenização.
- Previsões possuem incerteza e podem divergir das condições observadas.
- Em uma implantação pública seriam necessários autenticação, autorização, servidor WSGI, monitoramento e homologação das regras.

14. Conclusão

O Protege Antes demonstra o fluxo solicitado desde a obtenção de dados externos até a comunicação personalizada, incluindo uma entrega real por WhatsApp confirmada e auditada. A arquitetura multiagente deixa coleta, processamento, decisão e comunicação claramente separadas. O uso local do modelo reduz dependência externa, enquanto o Groq oferece uma alternativa remota mediante configuração de credencial. A interface e a auditoria tornam as decisões compreensíveis e adequadas a uma apresentação didática.

Projeto licenciado sob MIT. A entrega inclui o relatório técnico em PDF e o código-fonte com README, configuração de exemplo, migrações e instruções de execução. O arquivo `.env`, bancos locais, caches e credenciais não integram o pacote.